

Evaluación Parcial 2

Encargo con Defensa Técnica

Docente

Sigla	Nombre Asignatura	Tiempo Asignado	% Ponderación
DSY1103	Desarrollo FullStack 1	2semanas/15min	45%

1. Situación Evaluativa 1:

Ejecución práctica	X	Entrega de encargo con defensa	X	Presentación / Defensa Técnica		Prueba escrita (formato quiz)
--------------------	---	--------------------------------	---	--------------------------------	--	-------------------------------

2. Instrucciones generales

Descripción
- La Evaluación Sumativa 2 corresponde al primer avance técnico formal del Proyecto Semestral de Arquitectura de Microservicios, el cual será desarrollado en equipos de 2 a 3 integrantes y tiene como objetivo construir una solución completa basada en un dominio real o simulado, seleccionado preferentemente por los/as estudiantes (o asignado por el/la docente cuando corresponda). - Cada equipo deberá diseñar y desarrollar todos los microservicios necesarios para cubrir de manera integral las operaciones, reglas de negocio y entidades del dominio seleccionado. Como mínimo, se deben implementar 10 microservicios por equipo.

- No existe máximo, siempre que la arquitectura mantenga coherencia, cohesión y separación funcional.
- El tiempo asignado para el desarrollo de esta evaluación en el **taller de alto cómputo** es de **cuatro bloques pedagógicos**.
- El encargo será asignado en la **semana 1** y deberá ser **entregado en la semana 10**, antes de la primera sesión de evaluación. La **presentación/defensa técnica se realizará en las semanas 11 y 12**, con una duración máxima de **15 minutos por estudiante**. La defensa técnica se organizará **al azar**, por lo que todos los equipos deben estar preparados desde el inicio del período de evaluación.
- Aunque el proyecto se trabaja en **grupos**, la **defensa es individual**. Cada estudiante será evaluado de forma personal, en base a su **desempeño, claridad y dominio del proyecto**. La nota dependerá exclusivamente de lo que cada uno demuestre en la defensa técnica, por lo que es fundamental conocer a fondo todos los aspectos del desarrollo.
- Si durante la defensa un/a estudiante **no puede explicar con claridad el proyecto, no logra ejecutar correctamente la aplicación o no sabe modificar el código de forma funcional**, se asumirá que **no participó activamente en el desarrollo**.
- Esto afectará directamente su calificación, que será asignada **según su propio desempeño, independientemente de que el proyecto esté bien construido** o que su compañero/a lo haya defendido correctamente.
- Informar a los/las estudiantes que, al ser un encargo, deberán realizarlo previamente en su tiempo de trabajo autónomo

Propósito de la evaluación:

El propósito de esta evaluación es verificar la capacidad del equipo para diseñar, construir y justificar una arquitectura distribuida basada en microservicios independientes, con persistencia real, reglas de negocio claras, validaciones robustas y manejo adecuado de errores, demostrando dominio en:

- La estructuración de proyectos bajo el patrón CSR.
- La manipulación de base de datos
- El manejo adecuado de validaciones y errores.
- El desarrollo de endpoints REST completos y funcionales.
- El cumplimiento de buenas prácticas de organización y calidad de código.
- La implementación de herramientas de trabajo colaborativo.
- La integración de manejo de respuestas eficientes.

Instrucciones específicas de la Evaluación:

Consideraciones para la realización de la ejecución práctica:

Se recomienda realizar esta evaluación en laboratorio o sala equipada con computadores, emuladores o dispositivos físicos configurados, acceso a IntelliJ IDEA o VsCode, con las dependencias necesarias para Spring Boot configuradas, conexión a internet y visibilidad compartida de pantalla.

Requisitos obligatorios de la evaluación:

El desarrollo debe integrar todos los aprendizajes adquiridos durante la Experiencia de Aprendizaje 2, incluyendo:

- **Persistencia real con JPA + Hibernate**
 - Implementación de entidades con anotaciones como @Entity, @Id, @GeneratedValue, @OneToMany, @ManyToOne, etc.
 - Uso de repositorios mediante JpaRepository para ejecutar operaciones CRUD reales.
 - Configuración del datasource y dialecto en application.properties.
 - Creación y/o ejecución de migraciones iniciales de base de datos (scripts SQL o Flyway/Liquibase).
- **Modelado de datos y normalización básica**
 - Diseño de modelos relacionales coherentes con el dominio del proyecto.
 - Definición de relaciones correctas entre tablas y entidades.
 - Manejo de claves primarias, claves foráneas e integridad referencial.
- **Validaciones con Bean Validation (JSR 380)**
 - Uso de anotaciones
 - Validación de datos en controladores, retornando respuestas consistentes ante entradas inválidas.
 - Separación entre DTOs y entidades para validar datos de manera limpia y segura.
- **Manejo de excepciones básicas y respuestas controladas**
 - Uso de try/catch cuando corresponda en la capa de servicio.
 - Retorno de respuestas coherentes con ResponseEntity, utilizando códigos HTTP adecuados
 - Uso de @ControllerAdvice para manejo centralizado de errores.

- **Estructuración del código bajo el patrón CSR**
 - Separación clara entre:
 - Controller → manejo de solicitudes REST
 - Service → lógica de negocio
 - Repository → acceso a datos
 - Flujo ordenado, coherente y sin mezclar responsabilidades.
 - Uso de DTOs para comunicación entre capas cuando sea apropiado.
- **Respuestas REST correctas y consistentes**
 - Todos los endpoints deben retornar JSON estructurado.
 - Los microservicios deben aplicar buenos principios REST:
 - rutas semánticas
 - métodos HTTP adecuados (GET, POST, PUT, DELETE)
 - parámetros, path variables y request bodies organizados
 - Uso obligatorio de ResponseEntity para controlar la respuesta.
- **Integración de logs estructurados con SLF4J**
 - Mensajes claros que permitan trazabilidad entre capas.
 - Reflejo de eventos relevantes como creación, actualización, errores y validaciones fallidas.
- **Comunicación entre microservicios**
 - Consumo de endpoints entre microservicios mediante:
 - WebClient
 - o Feign Client
 - Manejo de timeouts, errores y validación de datos recibidos.
 - Pruebas de integración mínimas mediante Postman u otra herramienta REST.
- **Buenas prácticas y calidad de código**
 - Código comentado cuando sea necesario.
 - Nombres de clases, métodos y variables significativos.
 - Estructura limpia y modular del proyecto.

- Uso adecuado de paquetes según responsabilidad.
- Eliminación de código muerto o duplicado antes de la entrega.
- **Integración de herramientas de trabajo colaborativo y gestión de versiones como Git**

Forma de entrega:

- El proyecto debe estar subido a un repositorio de GitHub público con acceso otorgado al docente y a los miembros del equipo.
- El repositorio debe contener:
 - Carpeta del proyecto
 - Archivo README.md con descripción del proyecto, nombres de los/las estudiantes, funcionalidades implementadas y pasos para ejecutar.
 - Commits bien diseñados, distribuidos y generados que expliquen de manera correcta cada uno de ellos (no usar textos o descripciones no técnicas).
 - **Si se detectan cambios así sean mínimos luego de la fecha de entrega en el repositorio por cualquiera de los integrantes del equipo y antes de su evaluación se les asignará un 1.0 de manera automática en la nota tanto grupal como individual.**
- Se subirá al **enlace de AVA grupal el código fuente desarrollado** (con los mismos elementos que se suben al repositorio) por un solo miembro del equipo. En caso de no entregar en AVA no se podrá realizar por ningún otro medio ni después de la fecha límite y se le generará un 1.0 tanto en la nota grupal como individual de manera automática.
- Se activará el enlace en **AVA individual escribiendo su nombre y apellido y número de equipo asignado**, así como el nombre de su aplicación o contexto. En caso de no activarlo en el plazo establecido no tendrá derecho a la defensa y se le asignará un 1.0 como calificación individual de manera automática (Si podrá acceder a la nota grupal en caso de haberlo activado)

Consideraciones para todos los estudiantes:

- La defensa técnica es individual.
- No se permite el uso de inteligencia artificial, herramientas automáticas de código o asistencia externa durante la sesión.
- El uso de internet debe limitarse estrictamente a la descarga de dependencias del proyecto Spring Boot.
- El/la docente monitoreará continuamente el trabajo del estudiante mediante observación directa o herramientas de control de laboratorio.

- Cualquier indicio de colaboración no autorizada o plagio implicará la anulación inmediata de la evaluación.
- Puedes solicitar repetir la pregunta si no la entiendes.
- Puedes anotar los pasos que vas a seguir.
- El/la docente evaluará tanto lo que haces como lo que explicas.
- Si algún recurso de la entrega del código no está implementado en la aplicación desarrollada, todos los ítems tanto grupales como individuales asociados a dicho elemento se calificarán de manera inmediata con un 0, esto incluye a los elementos donde se deben desarrollar o corregir errores en tiempo real durante la defensa.

3. Pauta de Evaluación Rúbrica

Categoría	% logro	Descripción niveles de logro
Muy buen desempeño	100%	Demuestra un desempeño destacado, evidenciando el logro de todos los aspectos evaluados en el indicador.
Desempeño aceptable	60%	Demuestra un desempeño competente, evidenciando el logro de los elementos básicos del indicador, pero con omisiones, dificultades o errores.
Desempeño incipiente	30%	Presenta importantes omisiones, dificultades o errores en el desempeño, que no permiten evidenciar los elementos básicos del logro del indicador, por lo que no puede ser considerado competente.
Desempeño no logrado	0%	Presenta ausencia o incorrecto desempeño.

Categorías de Respuesta

Indicador de Evaluación	Muy buen desempeño 100%	Desempeño aceptable 60%	Desempeño incipiente 30%	Desempeño no logrado 0%	Ponderación Indicador de Evaluación
Dimensión Entrega de Encargo (Grupal)					
IE 1.2.1 Estructura el microservicio aplicando el patrón CSR (Controller–Service–Repository/Model), con separación real de responsabilidades y paquetes por capa.	Estructura el proyecto por paquetes (controller, service, model/repository); el controller solo orquesta; el service concentra lógica de negocio; el repository/model gestiona datos; dependencias en dirección correcta.		Estructura el proyecto por paquetes (controller, service, model/repository); pero con algunos errores en implementación de funcionalidades propias de uno o dos paquetes.	Estructura el CSR parcial: existe separación, pero queda lógica en el controller o hay acoplamientos menores entre capas.	No estructura el patrón de diseño 3%
IE 2.1.1 Modela esquemas de base de datos normalizados y coherente con el dominio separados para cada microservicio, estructurando entidades JPA, relaciones y atributos conforme a principios de diseño relacional.	Modela completamente la base de datos del dominio, con entidades bien definidas, atributos coherentes, claves primarias y foráneas correctas, y relaciones normalizadas según buenas prácticas. Cada microservicio posee su propio modelo consistente.		Modela la mayoría de las entidades y relaciones, pero presenta inconsistencias menores (nombres, tipos o relaciones) que no afectan la ejecución general.	Modela entidades básicas, pero con relaciones incompletas, mala normalización o incoherencia en atributos que afectan la integridad del modelo.	No modela correctamente las entidades ni relaciones; la base de datos no es funcional o no corresponde al dominio. 2%
IE 2.1.2 Integra operaciones CRUD completas en los microservicios, conectando repositorios JpaRepository con endpoints REST	Implementa CRUD completo para todas las entidades; los endpoints funcionan correctamente, retornan JSON válidos y reflejan cambios reales en la BD.		Implementa CRUD en la mayoría de las entidades, con algunos errores menores o una entidad faltante.	Implementa CRUD incompleto o con errores en múltiples operaciones; inconsistencias entre BD y API.	El CRUD no funciona, no está implementado o no se conecta con la persistencia real. 3%

funcionales y retornos JSON coherentes						
IE 2.2.1 Implementa las reglas de negocio definidas para el dominio, garantizando que la aplicación respeta restricciones, flujos, procesos y requisitos funcionales completos del contexto del proyecto.	Implementa todas las reglas de negocio necesarias, su lógica es sólida y responde correctamente a casos reales del dominio.		Implementa la mayoría de reglas con pequeñas omisiones o errores menores.	Implementa reglas básicas, pero omite elementos importantes del dominio o las aplica incorrectamente.	No implementa reglas de negocio o son incorrectas.	4%
IE 2.2.2 Implementa validaciones con Bean Validation y relaciones entre entidades que aseguran integridad de datos.	Aplica correctamente validaciones en DTOs/entidades y controla errores con mensajes claros.		Incluye validaciones, pero algunas no funcionan o son insuficientes.	Validaciones mínimas y errores no controlados.	No implementa validaciones.	3%
IE 2.2.3 Integra relaciones entre entidades manteniendo coherencia e integridad referencial.	Relaciones bien creadas, con integridad referencial funcional, cascadas apropiadas y consultas coherentes.		Relaciones parcialmente correctas, con detalles menores de funcionamiento.	Relaciones incompletas, inconsistentes o mal aplicadas.	No implementa relaciones o están mal definidas.	2%
IE 2.3.1 Implementa manejo de excepciones en todos los endpoints del microservicio, garantizando respuestas coherentes y uniformes que cubren los errores esperados del dominio.	Maneja excepciones en todos los endpoints, con códigos HTTP correctos y mensajes estructurados.		Manejo adecuado, pero con inconsistencias menores.	Manejo limitado o poco claro.	No maneja excepciones.	3%
IE 2.3.2 Integra logs estructurados en puntos clave del flujo del microservicio,	Logs estratégicos, consistentes, útiles y bien ubicados.		Logs presentes, pero con exceso o falta de claridad.	Logs mal ubicados o insuficientes.	No registra logs.	2%

permitiendo trazabilidad de las operaciones según buenas prácticas.						
IE 2.4.1 Implementa comunicaciones necesarias entre microservicios según el contexto elegido, asegurando interoperabilidad total en los flujos que requieren información remota.	Comunicaciones WebClient/Feign bien implementadas, funcionales y alineadas al dominio.		La comunicación funciona parcialmente.	Comunicación con errores o incoherencias.	No implementa comunicación.	3%
IE 2.4.2 Configura endpoints REST que exponen o consumen datos remotos respetando convenciones REST y coherencia de datos.	Endpoints correctos, semánticos y consistentes con respuestas remotas.		Endpoints funcionales con detalles menores.	Endpoints incorrectos o incompletos.	No expone ni consume endpoints remotos.	2%
IE 2.5.1 Gestiona un repositorio GitHub ordenado, con commits técnicos, progresivos y distribuidos equitativamente entre los integrantes.	Historial con commits técnicos, frecuentes y equitativos.		Commits presentes, pero con desequilibrio entre integrantes.	Pocos commits o mal documentados.	No usa control de versiones correctamente.	2%
IE 2.5.2 Organiza tareas del proyecto en Trello u otra herramienta colaborativa, evidenciando roles y avance del equipo.	Tablero organizado con tareas claras, asignaciones y avance visible.		Tablero con organización parcial.	Tablero incompleto o sin asignaciones reales.	No utiliza planificación.	1%
Porcentaje Entrega de Encargo						30%
Dimensión Defensa						

IE 2.1.3 Justifica decisiones de modelado, analizando relaciones, normalización y consistencia entre tablas y entidades según el dominio seleccionado	Explica adecuadamente relaciones, normalización, atributos y decisiones de diseño, demostrando comprensión total del modelo.		Explica parcialmente el modelo, pero algunas decisiones no están justificadas o muestran vacíos conceptuales.	Explica superficialmente el modelo sin claridad sobre relaciones ni normalización.	No puede explicar ni justificar el modelo de datos.	4%
IE 2.1.4 Realiza modificaciones en vivo asegurando compilación, persistencia y funcionamiento en pruebas REST.	Realiza cambios en vivo (entidades, relaciones o CRUD), compila sin errores y demuestra persistencia correcta en pruebas REST.		Realiza cambios en vivo, pero requiere correcciones menores o asistencia; finalmente funciona.	Intenta realizar cambios, pero genera errores o inconsistencias que impiden persistencia confiable.	No puede realizar cambios funcionales en vivo o el proyecto falla.	6%
IE 2.2.4 Justifica la lógica de negocio implementada en la capa de servicio, demostrando comprensión del flujo interno del microservicio.	Explica claramente la lógica de la capa de servicio, flujo de decisiones y su impacto en la integridad del microservicio.		Explica parcialmente la lógica con pequeñas confusiones.	Explicación superficial, confusa o incompleta.	No puede explicar la lógica del servicio.	6%
IE 2.2.5. Realiza cambios en el código (añadir validaciones o reglas), verificando su efecto en la API mediante pruebas REST.	Agrega o ajusta validaciones/reglas en vivo con éxito y demuestra su funcionamiento en pruebas REST.		Realiza cambios con apoyo o corrige errores menores.	Realiza cambios incorrectos o incompletos.	No puede realizar cambios funcionales.	12%
IE 2.3.3 Interpreta logs, excepciones y errores generados por el microservicio, explicando el flujo y la	Interpreta correctamente logs y excepciones, identifica origen del error y propone solución.		Interpreta parcialmente los logs y errores.	Tiene dificultad para interpretar errores.	No puede explicar logs ni errores.	7%

trazabilidad entre capas.						
IE 2.3.4 Realiza cambios en el código demostrando control adecuado de logs, excepciones y códigos HTTP.	Ajusta excepciones, logs o códigos HTTP en vivo con éxito y demuestra su efecto inmediatamente.		Lo ajusta con asistencia o correcciones menores.	Cambios incompletos o incorrectos.	No puede realizar cambios funcionales.	13%
IE 2.4.3 Explica el flujo de comunicación entre microservicios demostrando la correcta obtención, uso de información remota, justificando su manejo y su impacto en el flujo del negocio.	Explica claramente petición remota, respuesta, transformaciones y uso interno.		Explicación parcial con algo de confusión.	Explicación superficial.	No puede explicar la comunicación.	5%
IE 2.4.4 Realiza cambios en la comunicación entre microservicios, demostrando la correcta obtención y uso de información remota.	Modifica llamadas remotas en vivo correctamente y demuestra funcionamiento en pruebas.		Cambios funcionales con correcciones pequeñas.	Cambios incompletos o con errores.	No puede modificar la comunicación.	12%
IE 2.5.3 Explica su aporte personal al proyecto, justificando commits, tareas asignadas y participación individual en el desarrollo del proyecto semestral.	Presenta claridad total sobre su trabajo, commits, roles y aportes individuales.		Explica parcialmente su participación.	Explicación vaga o poco estructurada.	No puede explicar su aporte.	5%
Porcentaje Defensa						70%
Total						100%

